JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class AbstractMap.SimpleImmutableEntry<K,V>

java.lang.Object
  java.util.AbstractMap.SimpleImmutableEntry<K,V>

Type Parameters:

K - the type of the keys

V - the type of the value

All Implemented Interfaces:

Serializable, Map.Entry<K,V>

Enclosing class:

AbstractMap<K,V>

```
public static class AbstractMap.SimpleImmutableEntry<K,V>
extends Object
implements Map.Entry<K,V>, Serializable
```

An unmodifiable Entry maintaining a key and a value. This class does not support the `setValue` method. Instances of this class are not associated with any map nor with any map's entry-set view.

API Note:

Instances of this class are not necessarily immutable, as the key and value may be mutable. An instance of *this specific class* is unmodifiable, because the key and value references cannot be changed. A reference of this *type* may not be unmodifiable, as a subclass may be modifiable or may provide the appearance of modifiability.

This class may be convenient in methods that return thread-safe snapshots of key-value mappings. For alternatives, see the `Map::entry` and `Map.Entry::copyOf` methods.

Since:

1.6

See Also:

Serialized Form

Constructor Summary

Constructors

Constructor	Description
`SimpleImmutableEntry(Map.Entry<? extends K, ? extends V> entry)`	Creates an entry representing the same mapping as the specified entry.

SimpleImmutableEntry(K key, V value)	Creates an entry representing a mapping from the specified key to the specified value.

Method Summary

All Methods | Instance Methods

Modifier and Type	Method	Description
boolean	**equals**(Object o)	Compares the specified object with this entry for equality.
K	**getKey**()	Returns the key corresponding to this entry.
V	**getValue**()	Returns the value corresponding to this entry.
int	**hashCode**()	Returns the hash code value for this map entry.
V	**setValue**(V value)	Replaces the value corresponding to this entry with the specified value (optional operation).
String	**toString**()	Returns a String representation of this map entry.

Methods declared in class Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Constructor Details

SimpleImmutableEntry

```
public SimpleImmutableEntry(K key,
                            V value)
```

Creates an entry representing a mapping from the specified key to the specified value.

Parameters:

`key` - the key represented by this entry

`value` - the value represented by this entry

SimpleImmutableEntry

```
public SimpleImmutableEntry
(Map.Entry<? extends K, ? extends V> entry)
```

Creates an entry representing the same mapping as the specified entry.

Parameters:

`entry` - the entry to copy

Method Details

getKey

```
public K getKey()
```

Returns the key corresponding to this entry.

Specified by:

`getKey` in interface `Map.Entry<K,V>`

Returns:

the key corresponding to this entry

getValue

```
public V getValue()
```

Returns the value corresponding to this entry.

Specified by:

`getValue` in interface `Map.Entry<K,V>`

Returns:

the value corresponding to this entry

setValue

```
public V setValue(V value)
```

Replaces the value corresponding to this entry with the specified value (optional operation). This implementation simply throws

`UnsupportedOperationException`, as this class implements an unmodifiable map entry.

Specified by:

`setValue` in interface `Map.Entry<K,V>`

Implementation Requirements:

The implementation in this class always throws `UnsupportedOperationException`.

Parameters:

`value` - new value to be stored in this entry

Returns:

(Does not return)

Throws:

`UnsupportedOperationException` - always

equals

`public boolean equals(Object o)`

Compares the specified object with this entry for equality. Returns `true` if the given object is also a map entry and the two entries represent the same mapping. More formally, two entries `e1` and `e2` represent the same mapping if

```
(e1.getKey()==null ?
 e2.getKey()==null :
 e1.getKey().equals(e2.getKey()))
&&
(e1.getValue()==null ?
 e2.getValue()==null :
 e1.getValue().equals(e2.getValue()))
```

This ensures that the `equals` method works properly across different implementations of the `Map.Entry` interface.

Specified by:

`equals` in interface `Map.Entry<K,V>`

Overrides:

`equals` in class `Object`

Parameters:

`o` - object to be compared for equality with this map entry

Returns:

`true` if the specified object is equal to this map entry

See Also:

`hashCode()`

hashCode

`public int hashCode()`

Returns the hash code value for this map entry. The hash code of a map entry e is defined to be:

```
(e.getKey()==null   ? 0 : e.getKey().hashCode()) ^
(e.getValue()==null ? 0 : e.getValue().hashCode())
```

This ensures that `e1.equals(e2)` implies that `e1.hashCode()==e2.hashCode()` for any two Entries `e1` and `e2`, as required by the general contract of `Object.hashCode()`.

Specified by:

`hashCode` in interface `Map.Entry<K,V>`

Overrides:

`hashCode` in class `Object`

Returns:

the hash code value for this map entry

See Also:

`equals(Object)`

toString

`public String toString()`

Returns a String representation of this map entry. This implementation returns the string representation of this entry's key followed by the equals character ("=") followed by the string representation of this entry's value.

Overrides:

`toString` in class `Object`

Returns:

a String representation of this map entry

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.

Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.